

41026 Advanced Software Development

Prediction Market Intelligence Hub

An Agentic AI Microservices Application for Prediction Market Analysis

Assessment 1: Release 0: Agentic AI Foundations, Microservices & DevOps (20%)

Technical Report

Team Name: Group 29

Workshop Class / Tutor: Grace

Student #	Full Name	Student ID	GitHub Username	Assigned Feature
Student 1	Alex	26255859	AlexChoi2004	Market Watchlist & Discovery
Student 2	Christopher Kumarelil Cherian	25491198	christopherkumarelilcherian-bit	Portfolio & Position Tracker
Student 3	Syed Humza Hussain	25474457	SHumzaHussain/Silv3r-exe	Market Research Notes & News Feed
Student 4	Erfanur Rahman	24951459	Erfanur1	AI Market Analyst Assistant

GitHub Repository URL: <https://github.com/Erfanur1/ADVSD>

Submission Date: 6 September 2026

1. Project Overview

1.1 Introduction

Prediction Market Intelligence Hub is an Agentic AI-powered microservices application that helps users track, research, and analyse prediction markets (in the style of Polymarket or Kalshi), covering watchlisted markets, position tracking, research notes, and AI-assisted market analysis, through a single unified web portal. The application follows a microservices architecture in which each of the four team members independently owns a frontend, backend/API, and database microservice for one feature, with all features integrated behind a shared entry point. All market data used in development and demonstration is mock/historical data seeded into each database, rather than a live third-party feed, to keep local, CI, and cloud demonstrations reliable.

1.2 Problem Statement

Prediction markets like Polymarket and Kalshi let users trade on real-world outcomes, but tracking multiple markets, researching why prices move, and evaluating one's own predictive accuracy currently requires manually juggling several disconnected tools. There is no single place for a user to discover relevant markets, keep notes on their reasoning, track their own positions, and get AI-assisted analysis of market movements. Prediction Market Intelligence Hub addresses this by providing one unified, AI-assisted platform for market discovery, research, portfolio tracking, and analysis.

1.3 Project Objectives

- ☐ Provide a single, unified portal for tracking, researching, and analyzing prediction markets.
- ☐ Demonstrate a working microservices architecture with four independent feature teams.
- ☐ Integrate an Agentic AI layer (AI-Mode via Ollama) to assist users with natural-language market analysis queries.
- ☐ Apply the Plan → Act → Observe → Adapt agentic loop to the software development process itself.
- ☐ Establish CI/CD automation and containerisation as the foundation for Releases 1 and 2.
- ☐ Use seeded mock/historical market data throughout, so local, CI, and cloud demonstrations do not depend on a live third-party market API.

1.4 Individual Feature Allocation

Student	Feature	Brief Description
Student 1	Market Watchlist & Discovery	Lets users browse/search markets by category (politics, sports, crypto, economics) and maintain a personal watchlist of tracked markets.
Student 2	Portfolio & Position Tracker	Lets users log hypothetical or real positions (side, entry price, size) and track profit/loss and exposure by category over time.
Student 3	Market Research Notes & News Feed	Lets users attach research notes and tagged evidence to a specific market, and browse a feed of news snippets linked to that market.

Student 4	AI Market Analyst Assistant	Provides an AI chat interface that explains price movements and summarises notes/news for a market; each analysis run is saved as an editable/deletable Analysis Report record (the feature's core CRUD entity), and the assistant flags potentially mispriced or highly volatile markets.
-----------	-----------------------------	--

2. Project Analysis and Planning

2.1 Agile Team Project Plan

The team follows a lightweight Scrum-inspired approach across three sprints within Release 0 (see Section 2.2 backlog). Given the short release timeframe, the team coordinates via Discord for daily updates rather than formal daily standups: task claiming, progress updates, and blockers are posted asynchronously as they arise.

Sprint structure: three informal sprints, Sprint 1 (scaffolding and core CRUD), Sprint 2 (AI-Mode integration and remaining CRUD), Sprint 3 (integration, testing, and demo prep), aligned to the backlog in Section 2.2.

Ceremonies: sprint planning happens collaboratively when the backlog is created and tasks are claimed; retrospectives are informal, discussed in Discord at the end of each sprint before starting the next.

Tools: GitHub (shared repository, branches, pull requests), GitHub Actions (CI/CD), Discord (day to day coordination and task claiming), Docker Compose (local integration).

Roles: each student owns one feature end to end (frontend, backend, database, CI workflow, and their own report sections). Shared and integration responsibilities (Docker Compose, shared home page, architecture diagrams, report coordination) are distributed across the team via a claimed task list rather than fixed roles, to keep workload even.

2.2 Product Backlog

The full Release 0 backlog is listed below, grouped by feature so each student can see and work through their own user stories directly. A Sprint column shows which of the team's three sprints each story is planned for (Sprint 1: Weeks 1–2 scaffolding & core CRUD; Sprint 2: Weeks 3–4 AI-Mode integration & remaining CRUD; Sprint 3: Week 5 polish, testing, integration & demo prep). Update Sprint and Status as work is assigned and progresses; add, remove, or re-split stories as your team's actual plan requires.

Student 1 — Market Watchlist & Discovery

ID	User Story	Sprint	Status
US-01	As a user, I want to browse a list of available markets so that I can discover ones I'm interested in.	Sprint 1	<i>Completed</i>
US-02	As a user, I want to filter/search markets by category (politics, sports, crypto, economics) so that I can find relevant ones faster.	Sprint 1	<i>Completed</i>
US-03	As a user, I want to add a market to my watchlist so that I can track it over time.	Sprint 1	<i>Completed</i>

US-04	As a user, I want to remove a market from my watchlist so that I can keep my list relevant.	Sprint 2	<i>Completed</i>
US-05	As a user, I want to view my full watchlist with current prices so that I can see everything I'm tracking at a glance.	Sprint 2	<i>Completed</i>
US-06	As a user, I want to ask the AI "what markets are trending right now?" so that I get a natural-language summary via AI-Mode.	Sprint 2	<i>Completed</i>

Student 2 — Portfolio & Position Tracker

ID	User Story	Sprint	Status
US-07	As a user, I want to log a new position (market, side, entry price, size) so that I can track my hypothetical trades.	Sprint 1	<i>Completed</i>
US-08	As a user, I want to view all my open and closed positions so that I can review my activity.	Sprint 1	<i>Completed</i>
US-09	As a user, I want to update a position (e.g. close it, adjust size) so that my records stay accurate.	Sprint 1	<i>Completed</i>
US-10	As a user, I want to delete a position so that I can remove mistakes or test entries.	Sprint 2	<i>Completed</i>
US-11	As a user, I want to see my total P&L and exposure by category so that I understand my risk.	Sprint 2	<i>Completed</i>
US-12	As a user, I want to ask the AI to summarise my portfolio performance so that I get an easy-to-read overview via AI-Mode.	Sprint 2	<i>Completed</i>

Student 3 — Market Research Notes & News Feed

ID	User Story	Sprint	Status
US-13	As a user, I want to create a research note on a specific market so that I can record my reasoning.	Sprint 1	<i>Completed</i>
US-14	As a user, I want to view all notes attached to a market so that I can review my past research.	Sprint 1	<i>Completed</i>
US-15	As a user, I want to edit a note so that I can update it as new information comes in.	Sprint 1	<i>Completed</i>
US-16	As a user, I want to delete a note so that I can remove outdated research.	Sprint 2	<i>Completed</i>
US-17	As a user, I want to see a feed of news snippets linked to a market so that I stay informed without leaving the app.	Sprint 2	<i>Completed</i>
US-18	As a user, I want to ask the AI to summarise my notes on a market so that I get a quick recap via AI-Mode.	Sprint 2	<i>Completed</i>

Student 4 — AI Market Analyst Assistant

ID	User Story	Sprint	Status
US-19	As a user, I want to chat with an AI assistant about a specific market so that I can understand what's driving its price.	Sprint 1	<i>Completed</i>
US-20	As a user, I want each analysis I run to be saved as a report so that I can revisit it later.	Sprint 1	<i>Completed</i>
US-21	As a user, I want to view my saved analysis reports so that I can track how my understanding of a market evolved.	Sprint 2	<i>Completed</i>
US-22	As a user, I want to delete an old analysis report so that I can keep my history relevant.	Sprint 2	<i>Completed</i>
US-23	As a user, I want the assistant to flag potentially mispriced or highly volatile markets so that I know where to focus attention.	Sprint 2	<i>Completed</i>

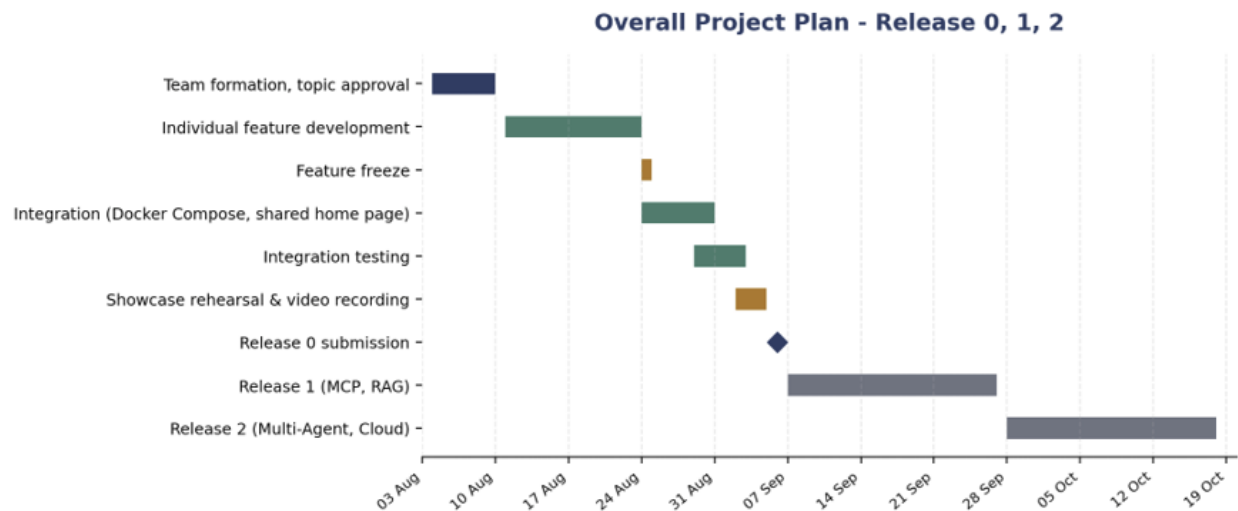
Team — Shared / Cross-Cutting Stories

ID	User Story	Sprint	Status
US-24	As a team, we want a unified home page (index.html) linking all 4 features so that the app feels like one product.	Sprint 1	<i>Completed</i>
US-25	As a team, we want a consistent shared CSS theme across all features so that the UI feels cohesive.	Sprint 1	<i>Completed</i>
US-26	As a team, we want one shared docker-compose.yml that runs all 11 containers together so that the whole app runs locally with one command.	Sprint 2	<i>Completed</i>
US-28	As a team, we want the Plan → Act → Observe → Adapt loop demonstrated in the terminal and logged in the report so that our agentic workflow is visible and verifiable.	Sprint 3	<i>Completed</i>
US-29	As a team, we want to document our prompt engineering process (iterations, context management) so that our AI development approach is transparent.	Sprint 3	<i>Completed</i>
US-31	As a team, we want a recorded 10-minute showcase video demonstrating every feature and the CI/CD pipeline so that our integrated app can be assessed.	Sprint 3	<i>Completed</i>

Individual — Per-Student DevOps & Testing Stories

ID	User Story	Sprint	Status
US-27	As a developer, I want my own GitHub Actions workflow (student-x.yml) so that my microservices are built and validated automatically on push.	Sprint 2	<i>Completed</i>
US-30	As a team, we want each database table seeded with 10+ records so that CRUD features are demonstrable with realistic data.	Sprint 3	<i>Completed</i>

2.3 Overall Project Plan



2.4 Functional & Non-Functional Requirements

Student	Functional Requirements	Non-Functional Requirements
Student 1	The system shall allow a user to browse/search markets, add/remove watchlist entries, and query the AI assistant for a trending-markets summary.	The AI-Mode endpoint shall return a graceful fallback message if AI-Mode is unreachable. Each database table shall be seeded with 10+ records on first run.
Student 2	The system shall allow a user to create, view, update, and delete prediction market portfolio positions via HTMX interactions and Flask backend endpoints	The Portfolio API shall respond within 500ms for 95% of standard CRUD requests, and gracefully handle timeouts up to 120 seconds when communicating with the shared AI-Mode service.
Student 3	The system shall allow a user to browse and search for a news feed, create/view/update/delete research notes linked to a specific article, and ask the AI assistant for research briefing summarizing recent news.	The AI-Mode endpoint shall return a graceful back message if AI-Modde is unreachable or returns an empty response; each dataset table shall be seeded with 10 or more records on first run.
Student 4	The system shall allow a user to browse/search prediction markets, ask the AI assistant to analyse a specific market for mispricing or chat freeform about a market, and create, view, update, and delete the resulting Analysis Report records.	The AI-Mode endpoint shall return a graceful fallback message ("AI-Mode is currently unavailable") if AI-Mode is unreachable or returns an empty response, rather than raising an error (verified by <code>test_ai_analyze_falls_back_when_ai_mode_unreachable</code> in <code>tests/test_api.py</code>). Each database table (markets, analyses, chat_messages) shall be seeded with 10+ records on the first run.

2.5 Individual Feature Plan

Student 1: Market Watchlist & Discovery

- Scope: allow users to browse and search available prediction markets, and maintain a personal watchlist of markets they want to track, with an optional note and priority per entry.
- Key screens: single-page frontend showing a live-searchable market list, a watchlist section (view, add, remove), and an AI-Mode panel for querying trending markets.
- Key endpoints: GET/POST/PUT/DELETE /markets; GET/POST/DELETE /watchlist; POST /ai/trending.
- Database schema summary: markets (id, title, category, current_probability, volume, close_date); watchlist (id, market_id FK, note, priority).
- Dependencies on other features: none. Watchlist & Discovery is self-contained and does not read from or write to any other student's database. Other features must call this feature's exposed API rather than access the markets database directly, per Section 7.1 of the specification.

Student 2: Portfolio and Position Tracker

- Scope: Implement a complete microservice triad (Frontend, Backend, Database) allowing users to manage portfolios, active positions, and trade logs while leveraging the shared AI container for risk analysis.
- Key screens: single-page frontend showing a live-searchable market list, a watchlist section (view, add, remove), and an AI-Mode panel for querying trending markets.
- Key endpoints: GET/ (Frontend UI Rendered via flask and htmx) GET /positions and POST /positions (List and create positions). DELETE /positions/<id> (Close/remove positions) POST /ai/analyse-risk (sends portfolio context to AI-mode).
- Database schema summary: Relational SQLite architecture utilizing three core tables (portfolios, positions, trade_history) pre-seeded with 10+ realistic prediction market entries.
- Dependencies on other features: Accesses the shared AI-mode service container running on port 8000 via internal Docker networking, remaining completely isolated from other features database layers.

Student 3: Market Research Notes and News Feed

- Scope: allow users to browse a feed of market related news articles and attach their own tagged research notes to a specific article with full CRUD on notes
- Key screens: a single frontend page showing a live searchable news feed, a research notes section (that has add/edit/delete, with title, content and tags) along with an AI-mode panel for requesting a brief of the research.
- Key endpoints: GET /news, GET /sources, GET/POST/PUT/DELETE /notes, POST /ai/briefing
- Database schema summary: sources (id, name, reliability_score) news_articles(id, headline, source_id FK, category, published_date, summary) research_notes(id, articleid FK, title, content, tags, created_at)
- Dependencies on other features: Market research notes and news feed is self contained and does not read or write to any other student database.

Student 4: AI Market Analyst Assistant

- Scope: allow users to browse prediction markets, run AI-assisted mispricing analysis or freeform chat about a specific market, and manage the resulting Analysis Report records (create/view/update/delete), with chat history retained per market.

- Key screens: single-page frontend (port 5104) showing a live-searchable market list, a saved-analyses panel (add, edit verdict, remove), and an "Ask the analyst" panel with two forms — analyse-for-mispricing and freeform chat — that render the AI's output and reasoning inline via HTMX.
- Key endpoints: GET/POST/PUT/DELETE /markets; GET/POST/PUT/DELETE /analyses; GET/DELETE /chat; POST /ai/analyze; POST /ai/chat.
- Database schema summary: markets (id, title, category, current_probability, volume, close_date); analyses (id, market_id FK, verdict, summary, confidence, created_at) — the feature's core CRUD entity; chat_messages (id, market_id FK nullable, role, content, created_at).
- Dependencies on other features: none. AI Market Analyst Assistant is self-contained and does not read from or write to any other student's database; it only calls the shared AI-Mode service (ai-mode:8000).

2.6 Individual Risk Management Plan

Risk	Likelihood / Impact	Owner	Mitigation
AI-Mode service unreachable during demo	Medium / Medium	Student 1	Implemented graceful fallback message instead of raw error (tested).
Local vs Docker hostname mismatches during dev	High / Low	Student 1	Standardised on .env files with python-dotenv across all services.
Generated files (DB, venv) accidentally committed	Medium / Low	Student 1	Added .gitignore rules (*.db, venv/, .env).
CI workflow silently not triggering (path mismatch)	Medium / Medium	Student 1	Diagnosed and fixed a repository structure issue; workflow confirmed passing.
Merge conflicts arising from team members modifying shared project scaffolds or root configurations simultaneously.	High / Medium	Student 2	Isolate all feature code strictly within the student-2/ directory, run continuous integration checks locally via pytest, and systematically pull main before pushing changes.
Communication timeouts or payload mismatches when connecting the Flask backend to the shared AI-Mode service container.	Medium / High	Student 2	Implement robust error handling and explicit JSON schema validations (task and context) matching the shared AI-Mode specifications.
SQLite database locks or concurrency errors during automated multi-service test runs.	Low / Medium	Student 2	Use isolated database file paths for individual test execution and manage connection lifecycles cleanly within context managers.

CI workflow failures on GitHub Actions due to missing dependencies or environment variables.	Medium / High	Student 2	Mirror the exact installation and test execution steps in student-2.yml locally before pushing code updates.
Silent data loss on note edits – the edit form only submitted the content field, so every Save silently wiped a note's title and tags back to blank	Medium/ Medium	Student 3	Rebuilt the edit form to submit title, content, and tags together. Verified with a before/after test that unrelated notes are unaffected.
Docker volume/ startup command misconfiguration causing Sqlite3.OperationalError: unable to open database file	Medium/ High	Student 3	Add the missing named volume (s3data) and corrected the container command to run init_db.py before the Flask app; verified via docker exec inspection of the container filesystem.
Shared theme changes accidentally breaking another student's page	Medium/ Medium	Student 3	Kept all shared CSS changes backward compatible with existing class names and structure, requested reviews from Student 1 and 4 via pull request before merging

2.7 Data Design

Each student's Database API owns its own SQLite schema exclusively; other services access this data only through the exposed API, never directly.

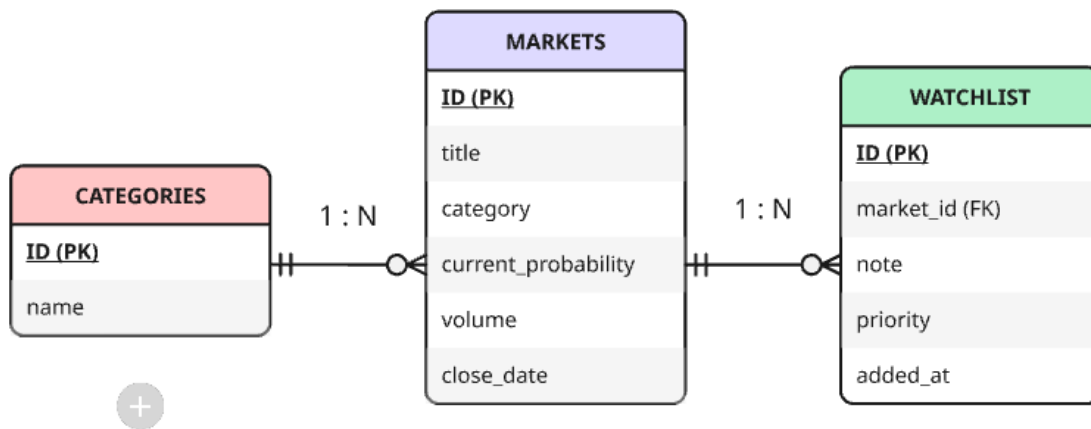
- ☐ Conceptual Data Model: [insert entities per feature, e.g. Market, Position, Note, Analysis Report]
- ☐ Entity Relationship Diagram (ERD): [insert per-feature ERD]
- ☐ Logical Data Model: [insert normalised table definitions]
- ☐ Physical Data Model: [insert actual SQLite CREATE TABLE definitions]

Student 1

Conceptual Data model:

- **Category:** represents a market topic grouping used for filtering (e.g. Politics, Sports, Crypto, Economics).
- **Market:** represents one prediction market a user can browse, search, and watch, including title, category, current implied probability, trading volume, and close date.
- **Watchlist Entry:** tracks a market the user has chosen to follow, with an optional note and a priority ranking.

ERD:



Logical Data Model:

- categories: normalised table mapping a unique primary key (id) to a unique category name (name).
- markets: normalised table holding one row per prediction market (id, title, category, current_probability, volume, close_date); category is stored as a text label matched against categories.name for filtering.
- watchlist: normalised relational table mapping market_id to its parent market, capturing an optional note, a priority ranking, and the timestamp the entry was added, enforcing cascade deletion so removing a market clears its watchlist entries.

Physical Data Model:

```

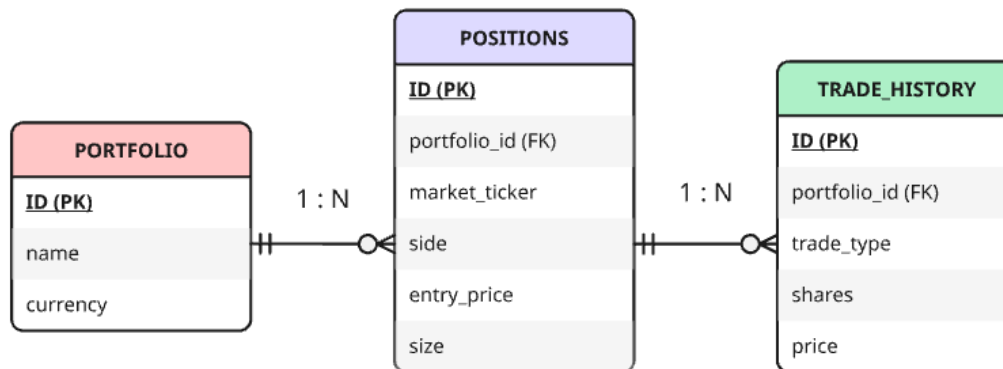
student-1 > database > schema.sql
1 CREATE TABLE IF NOT EXISTS categories (
2   id      INTEGER PRIMARY KEY AUTOINCREMENT,
3   name    TEXT NOT NULL UNIQUE
4 );
5
6 CREATE TABLE IF NOT EXISTS markets (
7   id      INTEGER PRIMARY KEY AUTOINCREMENT,
8   title   TEXT NOT NULL,
9   category TEXT NOT NULL,
10  current_probability REAL NOT NULL, -- 0..1
11  volume   INTEGER NOT NULL,
12  close_date TEXT NOT NULL
13 );
14
15 CREATE TABLE IF NOT EXISTS watchlist (
16   id      INTEGER PRIMARY KEY AUTOINCREMENT,
17   market_id INTEGER NOT NULL REFERENCES markets(id) ON DELETE CASCADE,
18   note    TEXT DEFAULT '',
19   priority INTEGER DEFAULT 0,
20   added_at TEXT DEFAULT CURRENT_TIMESTAMP
21 );
22
  
```

Student 2

Conceptual Data model:

- Portfolio: Represent financial grouping containers for user assets (portfolios).
- Position: Tracks active market stakes, share sizing and entry pricing tied to specific portfolios (positions)
- Trade History: Logs transactional audit records, share volumes and execution types for position events (trade_history).

ERD:



Logical Data Model:

- portfolios: Normalized table mapping a unique primary key (id) to a portfolio name (name) and base denomination (currency).
- positions: Normalized relational table mapping portfolio_id to parent portfolios while capturing market identifiers (market_ticker), trade directions (side), cost basis (entry_price), and holdings (size), enforcing cascade deletions.
- trade_history: Normalized transactional audit table mapping position_id to positions with detailed logs for action vectors (trade_type), units moved (shares), and execution costs (price), enforcing cascade deletions.

Physical Data Model:

```

CREATE TABLE IF NOT EXISTS portfolios (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  currency TEXT DEFAULT 'USD'
);

CREATE TABLE IF NOT EXISTS positions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  portfolio_id INTEGER,
  market_ticker TEXT NOT NULL,
  side TEXT NOT NULL,
  entry_price REAL NOT NULL,
  size INTEGER NOT NULL,
  FOREIGN KEY (portfolio_id) REFERENCES portfolios(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS trade_history (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  position_id INTEGER,
  trade_type TEXT,
  shares INTEGER,
  price REAL,
  FOREIGN KEY (position_id) REFERENCES positions(id) ON DELETE CASCADE
);

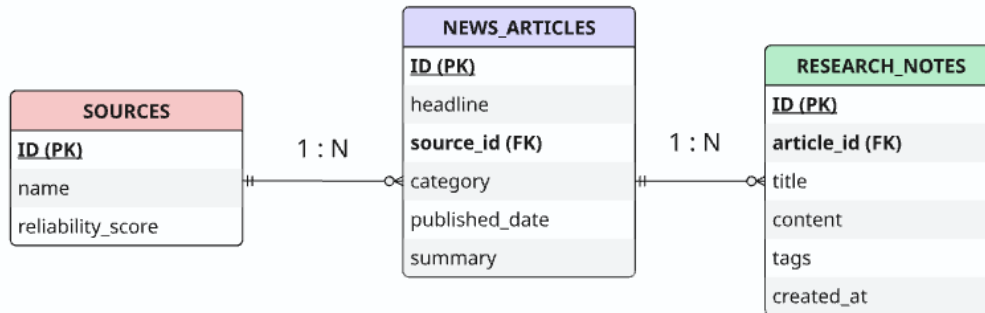
```

Student 3:

Conceptual Data Model:

- Source: Represents a news publisher and its reliability rating (eg, Reuters, TechCrunch)
- News Article: represents one news item a user can browse and search, linked to a source, with a category, publish date, and a summary
- Research note: a users own notes attached to a specific article, with a title, free-text content and comma-separated tags.

ERD:



Logical Data Model:

- **sources:** normalized table mapping a unique primary key (id) to a publisher name and a reliability score (0-1).
- **News_articles:** normalized table holding one row per article (id, headline, source_id, category, published_date, summary). Source_id is a foreign key to **sources**
- **research_notes:** normalized table mapping article_id to its parent article, capturing a title, content, tags, and creation timestamp; enforces cascade deletion so removing an article clears its notes.

Physical Data Model:

```

Release Notes: 1.136.0 X schema.sql X
student-3 > database > schema.sql
You, last week | 1 author (You)
1 CREATE TABLE IF NOT EXISTS sources (
2     id INTEGER PRIMARY KEY AUTOINCREMENT,
3     name TEXT NOT NULL UNIQUE,
4     reliability_score REAL NOT NULL -- 0..1
5 );
6
7 CREATE TABLE IF NOT EXISTS news_articles (
8     id INTEGER PRIMARY KEY AUTOINCREMENT,
9     headline TEXT NOT NULL,
10    source_id INTEGER NOT NULL REFERENCES sources(id) ON DELETE CASCADE,
11    category TEXT NOT NULL,
12    published_date TEXT NOT NULL,
13    summary TEXT NOT NULL
14 );
15
16 CREATE TABLE IF NOT EXISTS research_notes (
17     id INTEGER PRIMARY KEY AUTOINCREMENT,
18     article_id INTEGER NOT NULL REFERENCES news_articles(id) ON DELETE CASCADE,
19     title TEXT NOT NULL,
20     content TEXT DEFAULT '',
21     tags TEXT DEFAULT '',
22     created_at TEXT DEFAULT CURRENT_TIMESTAMP
23 );
24

```

Student 4

Conceptual Data Model:

- Market; a prediction market available for analysis (title, category, current implied probability, volume, close date).
- Analysis Report (analyses); the feature's core CRUD entity: one AI-generated or user-edited verdict/summary/confidence per market, timestamped, so a user can revisit how their read of a market evolved.
- Chat Message; one turn (user or assistant) in the AI analyst conversation, optionally scoped to a specific market.

Logical Data Model:

markets: normalised, one row per prediction market.

analyses: FK market_id → markets(id) ON DELETE CASCADE; verdict constrained to overpriced/underpriced/fair; confidence 0..1.

chat_messages: FK market_id → markets(id) ON DELETE SET NULL (a message is preserved even if its market is later removed); role constrained to user/assistant.

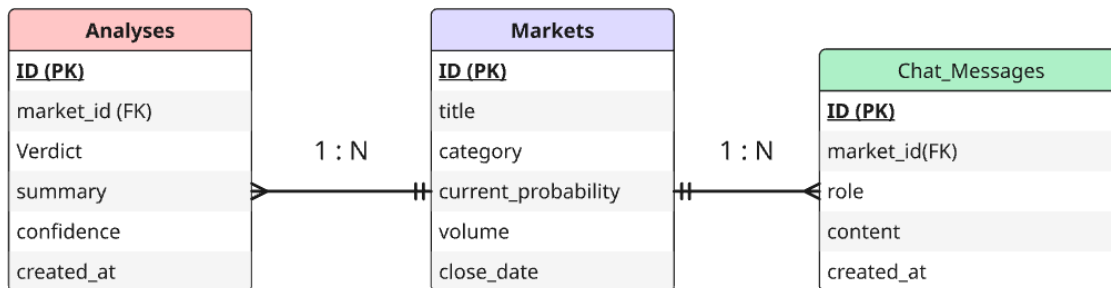
Physical Data Model:

```

Release Notes: 1.136.1 X schema.sql X
student-4 > database > schema.sql
1 CREATE TABLE IF NOT EXISTS markets (
2     id            INTEGER PRIMARY KEY AUTOINCREMENT,
3     title         TEXT NOT NULL,
4     category      TEXT NOT NULL,
5     current_probability REAL NOT NULL, -- 0..1
6     volume        INTEGER NOT NULL,
7     close_date    TEXT NOT NULL
8 );
9
10 CREATE TABLE IF NOT EXISTS analyses (
11     id            INTEGER PRIMARY KEY AUTOINCREMENT,
12     market_id     INTEGER NOT NULL REFERENCES markets(id) ON DELETE CASCADE,
13     verdict       TEXT NOT NULL, -- 'overpriced' | 'underpriced' | 'fair'
14     summary       TEXT NOT NULL,
15     confidence    REAL DEFAULT 0.5, -- 0..1
16     created_at    TEXT DEFAULT CURRENT_TIMESTAMP
17 );
18
19 CREATE TABLE IF NOT EXISTS chat_messages (
20     id            INTEGER PRIMARY KEY AUTOINCREMENT,
21     market_id     INTEGER REFERENCES markets(id) ON DELETE SET NULL,
22     role          TEXT NOT NULL, -- 'user' | 'assistant'
23     content       TEXT NOT NULL,
24     created_at    TEXT DEFAULT CURRENT_TIMESTAMP
25 );
26

```

ERD:



3. Repository Structure

The project follows the standard ASD 2026 repository structure with a shared microservice, four student microservice directories, shared AI services, and DevOps automation.

```

prediction-market-hub/
├── .github/workflows/  # student-1.yml ... student-4.yml
├── docs/              # architecture diagrams, reports (release-0/1/2)
├── shared/            # shared microservice: main UI, CSS theme
└── frontend/          # Home page (port 8080), css/theme.css

```

```
|— student-1/ ... student-4/ # each: frontend/ backend/ database/ tests/ Dockerfile
|— ai-services/      # ai-mode/, mcp-server/, rag-server/, multi-agent-server/
|— scripts/         # build/ test/ deploy/
|— docker-compose.yml
|— README.md
|— .gitignore
```

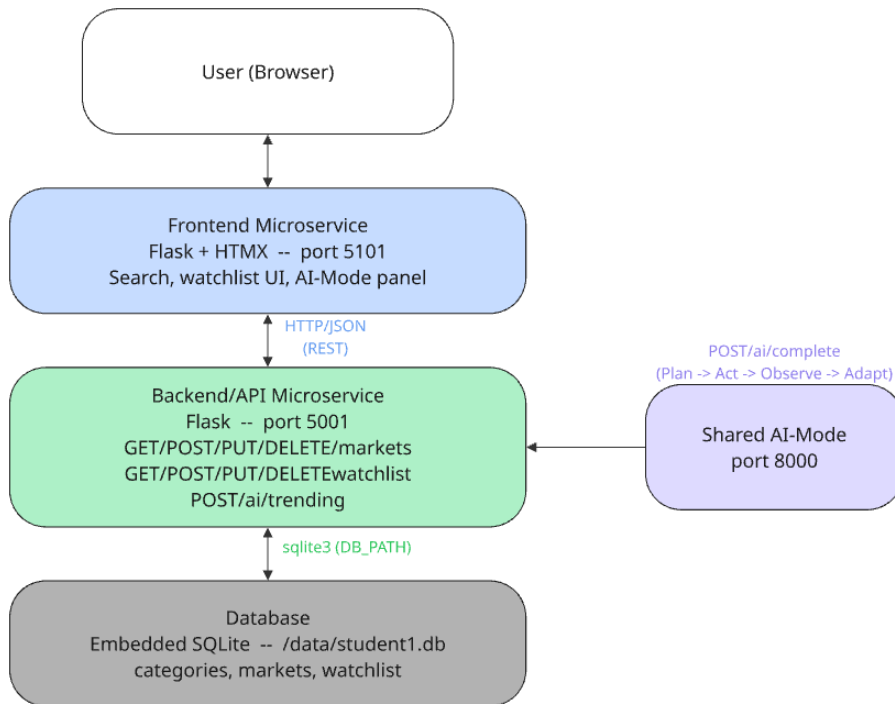
4. Individual Software Architecture

Each student's microservice set currently follows a two-tier pattern (Frontend, Backend/API with an embedded SQLite database), containerised independently and connected to the shared services (Ollama, AI-Mode, and the shared home page).

Student	Frontend	Backend / API	Database API
Student 1	Port 5101: Watchlist & discovery UI	Port 5001: Market search/watchlist logic + AI-Mode call	Embedded SQLite (markets, watchlist), via DB_PATH volume
Student 2	Port 5102: Portfolio UI	Port 5002: Position/P&L calculation + AI-Mode call	Embedded SQLite (positions, trades) - not yet wired to AI-Mode/volume
Student 3	Port 5103: Research notes & news UI	Port 5003: Notes/news logic + AI-Mode call	Embedded SQLite (notes, news_items), via DB_PATH volume
Student 4	Port 5104: AI analyst chat UI	Port 5004: Analysis orchestration + AI-Mode call	Embedded SQLite (analysis_reports), via DB_PATH volume

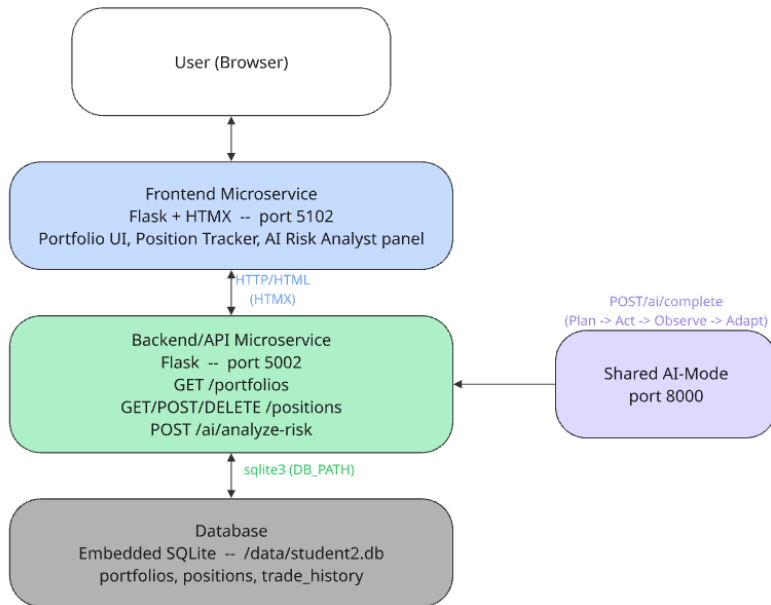
Student 1:

Student 1: Individual Software Architecture Market Watchlist & Discovery

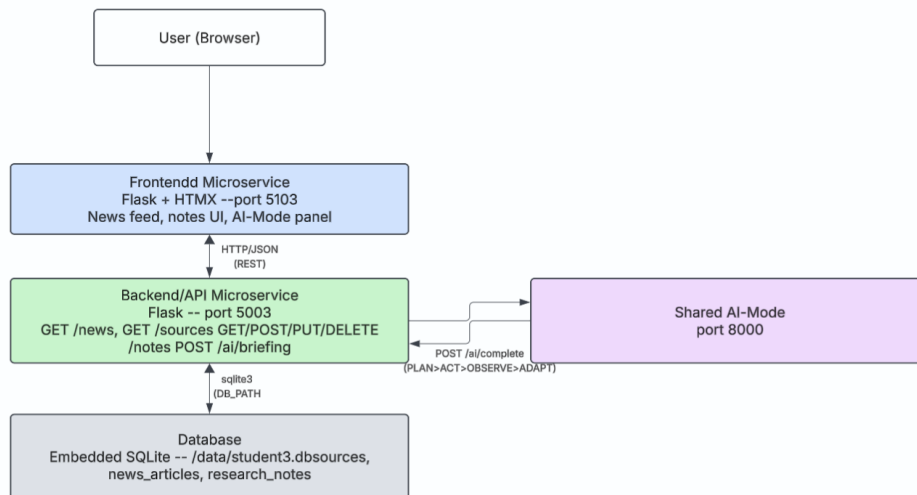


Student 2:

Student 2: Individual Software Architecture - Portfolio & Position Tracker



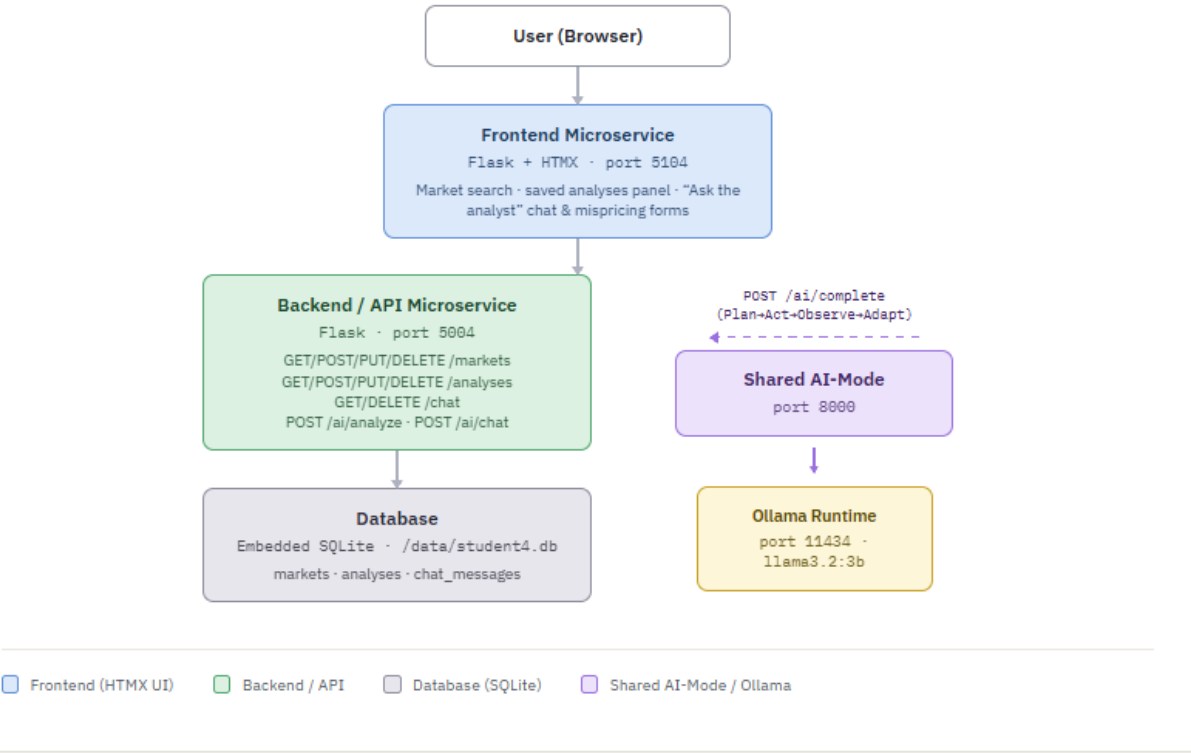
Student 3:



Student 4:

Student 4: Individual Software Architecture

AI Market Analyst Assistant · frontend 5104 · backend 5004

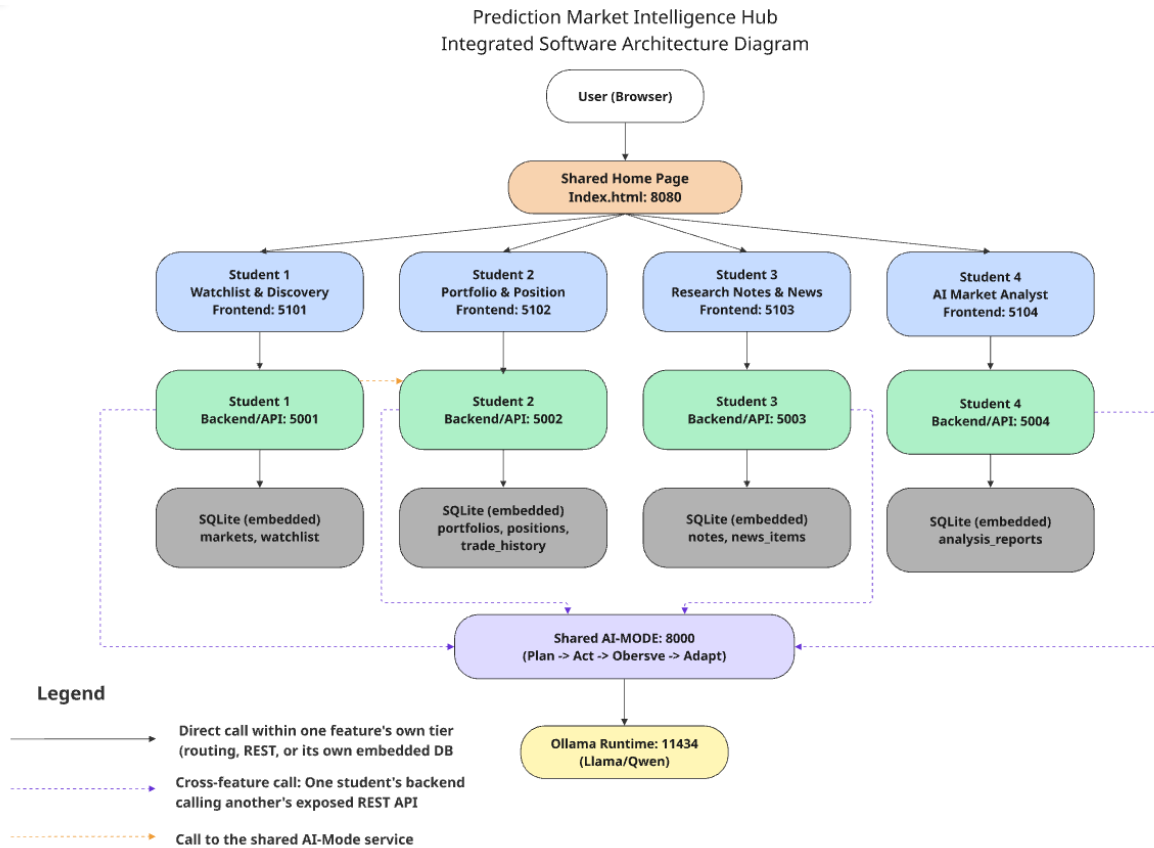


5. Integrated Software Architecture (Release 0)

The integrated architecture centres on a Shared Microservice, providing the Main UI Navigation, a Shared Access API, and an Access Database, which routes to each student's independently containerised frontend, backend/API, and database. Cross-feature data is never accessed directly; it is always retrieved through the owning student's exposed Database API (see example: Student 1's Backend/API calling student-2-db:6002 and student-3-db:6003 over HTTP).

Layer	Service	Port	Owner
Shared	Ollama runtime	11434	Team
Shared	AI-Mode	8000	Team
Shared	Home (shared frontend)	8080	Team
Student 1	Frontend / Backend	5101 / 5001	Student 1
Student 2	Frontend / Backend	5102 / 5002	Student 2
Student 3	Frontend / Backend	5103 / 5003	Student 3

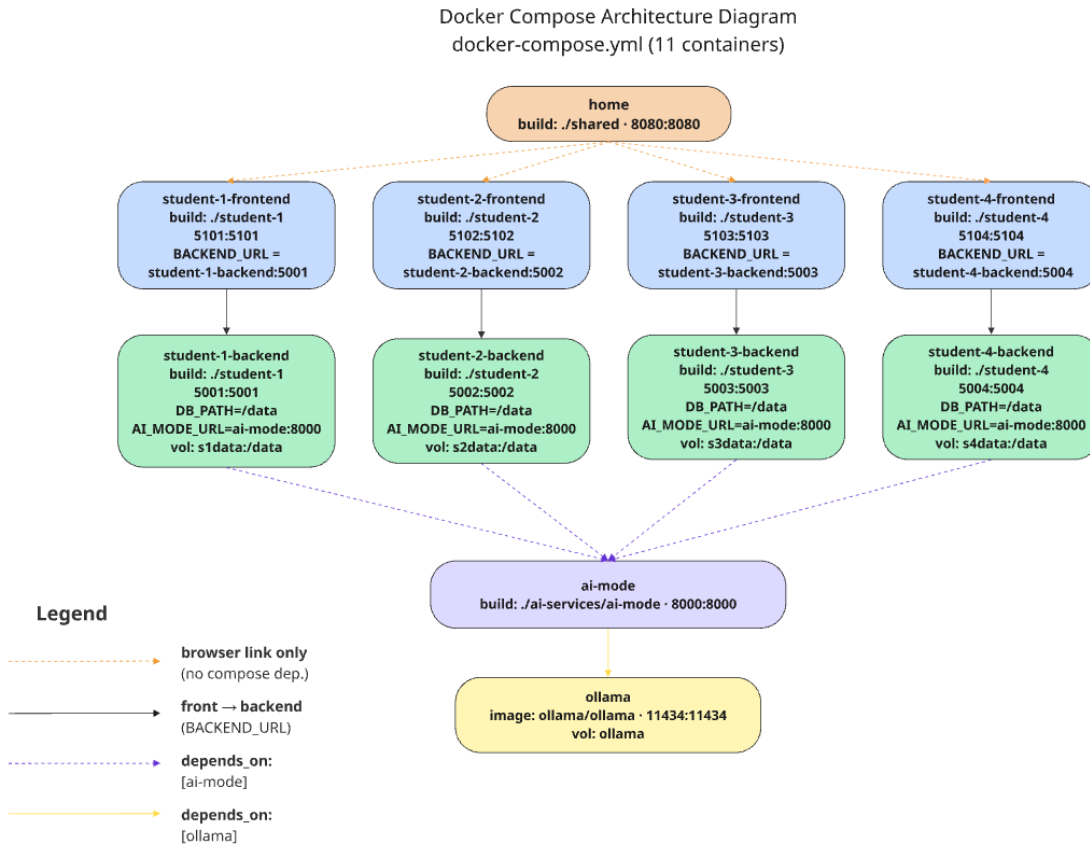
Student 4	Frontend / Backend	5104 / 5004	Student 4
-----------	--------------------	-------------	-----------



6. Docker Compose Architecture

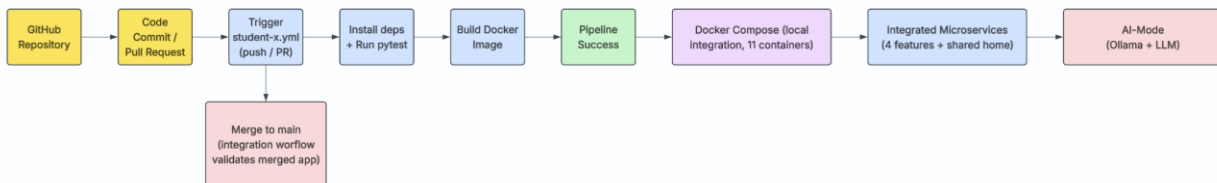
The application runs as an 11-container stack under a single docker-compose.yml: 2 shared infrastructure containers (Ollama, AI-Mode) + 1 shared home page + 4 students × 2 containers each (frontend, backend with embedded SQLite) = 11 containers.

- ❑ Each container is built from its own Dockerfile (student-x/Dockerfile, ai-services/ai-mode/Dockerfile, shared/Dockerfile).
- ❑ Environment variables define inter-service URLs (e.g. AI_MODE_URL=http://ai-mode:8000, BACKEND_URL=http://student-1-backend:5001).
- ❑ Named volumes (ollama, s1data, s2data, s3data, s4data) persist Ollama models and SQLite data across container restarts.
- ❑ All four backend services now declare depends_on: [ai-mode] and the matching DB_PATH / AI_MODE_URL environment variables.



7. DevOps Pipeline Architecture - HUMZA

- Trigger: push / pull_request to main
- Steps: checkout → set up Python/Node → install dependencies → lint → run unit tests → build Docker image → (Release 0: stop here; later releases add push/deploy steps)



8. Agentic AI Workflow

8.1 AI-Mode Implementation

AI-Mode is implemented using the Ollama runtime with approved open-source LLMs (Llama and Qwen). Each backend/API microservice calls AI-Mode following the workflow: Frontend → Backend/API → Ollama → LLM → Response.

8.2 Plan → Act → Observe → Adapt Loop

Stage	Description for Prediction Market Intelligence Hub
Plan	The agent interprets the user's natural-language request (e.g. "why did the price move on this market?") and plans which microservice/API call(s) are needed.
Act	The backend/API executes the planned action, e.g. queries the Market Research Notes & News Feed database API for recent notes and news linked to that market.
Observe	The result is checked against the original request (e.g. was enough recent context actually retrieved to explain the move?).
Adapt	If the result is unsatisfactory, the agent adjusts its plan, e.g. widens the lookback window or checks the Forecast Calibration service for related markets, and responds to the user.

Student 1 implementation note: For Watchlist & Discovery, the loop maps directly onto "what markets are trending right now?"

- Plan selects the top markets by trading volume from the markets table as context (rather than passing the whole table to the LLM)
- Act sends that context to the shared AI-Mode /ai/complete endpoint asking it to summarise which watchlisted markets are trending and why.
- Observe checks whether AI-Mode returned a non-empty summary.
- Adapt either returns that summary or, if AI-Mode is unreachable or times out, falls back to a plain-text message instead of crashing the request.

All four stages are appended to an `agentic_trace` list and returned alongside the AI output, so a marker can watch the /ai/trending response body (or the terminal logs) during the showcase and see each stage of the loop fire for this specific feature, not just a generic "it works" claim.

Student 2 implementation note: For the Portfolio & Position Tracker, the loop maps directly onto "Analyze My Portfolio Risk" via the /ai/analyse-risk endpoint.

- Plan gathers the user's current open investments from the positions table to build a targeted context for the LLM (rather than passing the entire database or irrelevant trade history).
- Act sends that precise context as a prompt to the shared AI-Mode /ai/complete endpoint (port 8000), asking it to evaluate the portfolio's exposure and generate a risk analysis.

- Observe waits and evaluates whether the shared AI service returned a valid, usable response or if the network connection timed out.
- Adapt either returns the generated risk analysis or, if AI-Mode is unreachable, gracefully falls back to a UI error state (e.g., "AI-Mode is currently unavailable") instead of crashing the backend container on port 5002.

All four stages are appended to an `agentic_trace` list and returned alongside the AI output. Because the frontend utilizes HTMX, this agentic trace is swapped directly into the DOM without a full page reload, allowing a marker to watch the `/ai/analyse-risk` response render each stage of the loop in real-time during the showcase.

Student 3 implementation note: For Market Research Notes and News feed the loop maps onto the "give me a research briefing" request.

- **Plan** selects the 6 most recently published articles from the `news_articles` table as context rather than sending the whole table to the LLM.
- **Act** sends that context to the shared AI mode `/ai/complete` endpoint, asking it to write a short research briefing summarising the news and what to watch.
- **Observe** checks whether AI-Mode returned a non empty response
- **Adapt** returns the generated briefing if successful or falls back to a plain-text "AI-Mode is currently unavailable" message if AI-Mode is unreachable or returns nothing avoiding a crashed request

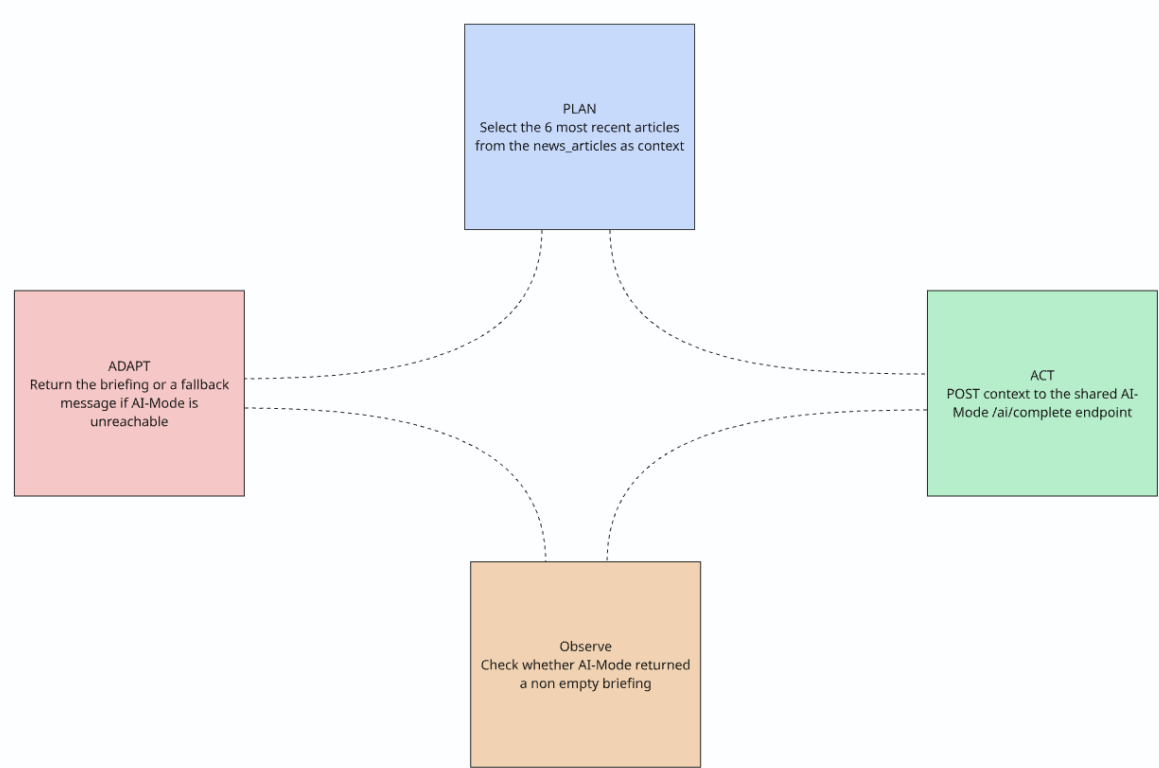
All four stages are appended to an `agentic_trace` list returned alongside the output, matching the pattern used across the other three features.

Student 4 implementation note: For the AI Market Analyst Assistant, the loop maps onto both the `/ai/analyze` (mispricing check) and `/ai/chat` (freeform Q&A) endpoints.

- Plan builds a compact context string for the requested market (title, category, current probability, volume, close date) rather than passing the whole markets table; for chat, it also pulls the last 4 messages of history for that market so the LLM has conversational grounding.
- Act sends that context plus a task string to the shared AI-Mode `/ai/complete` endpoint (`ai-mode:8000`).
- Observe checks whether AI-Mode was reachable and returned a non-empty output string.
- Adapt on success persists the result, `/ai/analyze` inserts a new row into `analyses`, `/ai/chat` inserts the assistant's reply into `chat_messages`, and returns it to the frontend; on failure it returns the plain-text fallback ("AI-Mode is currently unavailable. Please try again shortly.") instead of raising, so the HTMX frontend never breaks.
- Each stage is appended to an `agentic_trace` list returned alongside the output. `test_ai_analyze_falls_back_when_ai_mode_unreachable` in `tests/test_api.py` asserts the Plan/Act/Observe stages fire in order even when AI-Mode can't be reached, so the loop is verifiable outside the demo too.

8.3 Diagram

PLAN > ACT > OBSERVE > ADAPT LOOP
Example: Give me a research briefing (student 3)



9. GitHub Actions Workflows

To ensure code stability and enforce Continuous Integration (CI) across our microservices, each student maintains an independent GitHub Actions workflow file. These pipelines automatically trigger upon pushes or pull requests to the main branch, executing the following automated steps:

- Environment Setup: Spins up an Ubuntu runner and initializes the Python environment.
- Dependency Installation: Installs the required packages for both the Flask backend and the frontend.
- Automated Testing: Executes the Pytest suite to validate routing, database logic, and API endpoints.
- Docker Validation: Builds the Docker images to guarantee the containers will compile successfully in the production environment.

The specific workflow files for each microservice are outlined below:

Workflow File	Purpose
---------------	---------

student-1.yml	Build and validate the Market Watchlist & Discovery frontend, backend, and database services.
student-2.yml	Build and validate the Portfolio & Position Tracker frontend, backend, and database services.
student-3.yml	Build and validate the Market Research Notes & News Feed frontend, backend, and database services.
student-4.yml	Build and validate the AI Market Analyst Assistant frontend, backend, and database services.

10. Implementation Summary

Release 0 successfully delivered a fully functional Agentic AI microservices application. All four student features—Market Watchlist & Discovery, Portfolio & Position Tracker, Market Research Notes & News Feed, and AI Market Analyst Assistant—are functional end-to-end. Each microservice triad (Frontend, Backend, Database) operates independently while successfully integrating with the shared AI ecosystem.

Scope and Partial Features (MVP Status) For this Release 0 Minimum Viable Product (MVP), the primary focus was establishing stable infrastructure, network routing, and database integrity. The applications are currently seeded with mock data to demonstrate data flow. Extended CRUD capabilities—such as web forms for creating new user positions or viewing historical trade logs—have been successfully scoped and deferred to Release 1.

Key Technical Decisions & Deviations

- LLM Optimization:** The most significant deviation from the initial plan was our choice of language model. We transitioned from Llama 3.1 to the smaller llama3.2:3b model. This critical DevOps decision resolved severe Out-of-Memory (OOM) container crashes that occurred on local machines with 8GB of RAM, ensuring the ollama container remained stable.
- Microservice Isolation:** We strictly enforced separation of concerns by deploying 11 independent Docker containers orchestrated via docker-compose. Each backend utilizes an isolated, embedded SQLite database mapped to a local volume for persistence, utilizing strict 1NF relational models (e.g., ON DELETE CASCADE) to maintain data integrity.
- HATEOAS / HTMX Architecture:** Instead of building a heavy React frontend and JSON APIs, we opted for Flask backends serving HTML templates directly to HTMX frontends. This drastically reduced client-side JavaScript while allowing seamless, dynamic DOM updates for our Agentic AI outputs.
- Agentic Orchestration:** We successfully implemented a unified Plan -> Act -> Observe -> Adapt loop across all features. Rather than crashing upon network failures, the backends are engineered to catch connection timeouts with the shared ai-mode service and gracefully return UI error states.

11. GitHub Actions Workflow Evidence

Screenshots are in chronological order for each workflow:

Student-1-ci

16 workflow runs			Event ▾	Status ▾	Branch ▾	Actor ▾
This workflow has a workflow_dispatch event trigger.			Run workflow ▾			
✓	Merge pull request #10 from Erfanur1/feature-01/add-category	main	Sep 3, 8:10 PM GMT+10	24s	...	
student-1-ci #16: Commit f02ee9 pushed by AlexChoi2004						
✓	Fix search bar to match category as well as title	feature-01/add-category	Sep 3, 8:10 PM GMT+10	23s	...	
student-1-ci #15: Pull request #10 opened by AlexChoi2004						
✓	Fix search bar to match category as well as title	feature-01/add-category	Sep 3, 8:07 PM GMT+10	21s	...	
student-1-ci #14: Commit bb9cea3 pushed by AlexChoi2004						
✓	Merge pull request #6 from Erfanur1/feature/theme-redesign	main	Sep 2, 11:57 PM GMT+10	24s	...	
student-1-ci #13: Commit 76d833a pushed by ShumzaHussain						
✓	unified theme redesign	feature/theme-redesign	Sep 2, 11:56 PM GMT+10	25s	...	
student-1-ci #12: Pull request #6 opened by ShumzaHussain						
✓	unified theme redesign	feature/theme-redesign	Sep 2, 11:54 PM GMT+10	18s	...	
student-1-ci #11: Commit 987f1d4 pushed by ShumzaHussain						
✓	resolve merge conflicts with main	Christopher/Student-2/Relea...	Sep 2, 1:10 AM GMT+10	27s	...	
student-1-ci #10: Commit c14d814 pushed by christopherkumarellicherian-bit						
✓	Merge pull request #2 from Erfanur1/fix/student-1-workflow	main	Aug 26, 2:32 AM GMT+10	18s	...	
student-1-ci #9: Commit 12dcdd6 pushed by AlexChoi2004						
✓	Fix watchlist update form	fix/student-1-workflow	Aug 26, 2:31 AM GMT+10	25s	...	
student-1-ci #8: Pull request #2 opened by AlexChoi2004						
✓	Fix watchlist update form	fix/student-1-workflow	Aug 26, 2:27 AM GMT+10	21s	...	
student-1-ci #7: Commit c0cd2f1 pushed by AlexChoi2004						
✓	Merge pull request #1 from Erfanur1/fix/student-1-workflow	main	Aug 26, 2:16 AM GMT+10	25s	...	
student-1-ci #6: Commit 7027ca1 pushed by AlexChoi2004						
✓	Fix student-1 workflow paths	fix/student-1-workflow	Aug 26, 2:14 AM GMT+10	19s	...	
student-1-ci #5: Pull request #1 synchronize by AlexChoi2004						
✓	Trigger student-1 CI when workflow changes	fix/student-1-workflow	Aug 26, 2:14 AM GMT+10	24s	...	
student-1-ci #4: Commit ae54a46 pushed by AlexChoi2004						
✗	Fix repository structure and move workflow to root	main	Aug 26, 1:52 AM GMT+10	9s	...	
student-1-ci #3: Commit cde3670 pushed by AlexChoi2004						
✓	Add Update to watchlist frontend UI to complete CRUD	Alex/Student-1/Release-0	Aug 26, 12:48 AM GMT+10	22s	...	
student-1-ci #2: Commit 478bda9 pushed by AlexChoi2004						
✓	Trigger CI: confirm workflow runs after moving .github to repo root	Alex/Student-1/Release-0	Aug 26, 12:28 AM GMT+10	18s	...	
student-1-ci #1: Commit 9200a03 pushed by AlexChoi2004						

Brief Descriptions:

1. Trigger CI: Confirm workflow runs after moving .github to the repo root. Initial commit on Alex/Student-1/Release-0 confirmed workflow triggers correctly after relocating .github/workflows/ to the repository root.
2. Add Update to Watchlist: Updated the watchlist frontend UI to complete CRUD functionality. Added the missing inline "Update" action to the watchlist frontend, completing full CRUD (Create, Read, Update, Delete) support.
3. Fix Repository Structure: Fixed repository structure and moved the workflow to the correct route. An interim commit on main was made before the workflow path was corrected. This run failed because the workflow file was not yet at the expected path.

4. Trigger Student-1 CI When Workflow Changes: Added a paths filter on the fix/student-1-workflow branch so CI re-runs whenever student-1.yml itself is edited.
5. Fix Student-1 Workflow Paths: Corrected the paths filter in student-1.yml so CI reliably triggers on changes under student-1/.
6. Merge Pull Request #1: Merged pull request #1 (fix/student-1-workflow) into main, incorporating the corrected workflow paths filter.
7. Attempt to Fix Watchlist Update Form: Made a first attempt at fixing a bug in the HTMX watchlist update form caused by a mismatched form field and endpoint. A follow-up fix was required.
8. Fix Watchlist Update Form: Completed the fix for the watchlist update form and opened pull request #2 on fix/student-1-workflow.
9. Merge Pull Request #2: Merged pull request #2 (fix/student-1-workflow) into main, incorporating the completed watchlist update-form fix.
10. Resolve Merge Conflicts: Resolved merge conflicts introduced when Student 2's branch (Christopher/Student-2/Release-0) was merged into main.
11. Unified Theme Redesign (Commit): Applied the team's shared CSS theme (theme.css) to the Watchlist and Discovery frontend, creating a unified theme and visual redesign.
12. Unified Theme Redesign (Pull Request): Opened pull request #6 for the shared theme redesign.
13. Merge Pull Request #6: Merged pull request #6 (feature/theme-redesign) into main, completing Release 0 styling for Student 1's feature.
14. Attempted Search Bar Fix: Attempted to update the search functionality so it matches both the market title and category. The first attempt extended the market search to include categories and was committed on the feature/01-add-category branch.
15. Fix Search Bar: Refined the search bar implementation so it correctly matches both category and title. Opened pull request #10.
16. Merge Pull Request #10: Merged pull request #10 (feature/01-add-category) into main, finalising the category browsing feature for Release 0.

Student-2-ci

 Merge pull request #9 from Erfanur1/fix/student-2-theme-consistency student-2-ci #6: Commit 91391a0 pushed by Erfanur1	main	 Today at 1:12 AM  31s	...
 Fix student-2 frontend theme inconsistency student-2-ci #5: Pull request #9 opened by Erfanur1	fix/student-2-theme-consist...	 Today at 1:11 AM  26s	...
 Fix student-2 frontend theme inconsistency student-2-ci #4: Commit d52079b pushed by Erfanur1	fix/student-2-theme-consist...	 Today at 1:11 AM  34s	...
 Merge pull request #7 from Erfanur1/fix/student-2-release-0 student-2-ci #3: Commit 8cb21ec pushed by Erfanur1	main	 Today at 12:21 AM  25s	...
 Fix student-2 (Portfolio & Position Tracker) Release 0 bugs student-2-ci #2: Pull request #7 opened by Erfanur1	fix/student-2-release-0	 Today at 12:20 AM  21s	...
 Fix student-2 (Portfolio & Position Tracker) Release 0 bugs student-2-ci #1: Commit eebb73d pushed by Erfanur1	fix/student-2-release-0	 Today at 12:09 AM  27s	...

Brief Description:

1. **Run #1 (Commit Push):** Triggered by a direct commit (eebb73d) pushed to the fix/student-2-release-0 branch. This run successfully validated initial bug fixes for the Portfolio & Position Tracker microservice.
2. **Run #2 (Pull Request Opened):** Triggered automatically when Pull Request #7 was opened to merge the Release 0 bug fixes. The pipeline passed, confirming that the isolated branch was safe to review.
3. **Run #3 (Branch Merge):** Triggered by the completion of Pull Request #7. This run validated the main branch immediately after the bug fixes were merged, ensuring production stability.
4. **Run #4 (Commit Push):** Triggered by a new commit (d52079b) pushed to a separate branch (fix/student-2-theme-consistency) dedicated to resolving frontend UI and styling issues.
5. **Run #5 (Pull Request Opened):** Triggered when Pull Request #9 was opened for the theme of consistency updates. The CI pipeline successfully tested the CSS/HTML changes.
6. **Run #6 (Branch Merge):** Triggered by the final merge of Pull Request #9 into the main branch. This confirmed that the theme updates were successfully integrated without breaking the existing backend routing or database logic.

Student-3-ci

✓ Merge pull request #6 from Erfanur1/feature/theme-redesign student-3-ci #7: Commit 76d833a pushed by SHumzaHussain	main	Sep 2, 11:57 PM GMT+10 22s	...
✓ unified theme redesign student-3-ci #6: Pull request #6 opened by SHumzaHussain	feature/theme-redesign	Sep 2, 11:56 PM GMT+10 18s	...
✓ unified theme redesign student-3-ci #5: Commit 987f14d pushed by SHumzaHussain	feature/theme-redesign	Sep 2, 11:54 PM GMT+10 27s	...
✓ resolve merge conflicts with main student-3-ci #4: Commit c14d814 pushed by christopherkumarellcherian-bit	Christopher/Student-2/Relea...	Sep 2, 1:10 AM GMT+10 22s	...
✓ Merge pull request #4 from Erfanur1/feature/student-3-release-0 student-3-ci #3: Commit 948a5c2 pushed by SHumzaHussain	main	Sep 1, 5:41 PM GMT+10 20s	...
✓ Student 3: Market Research Notes & News Feed (Release 0) student-3-ci #2: Pull request #4 opened by SHumzaHussain	feature/student-3-release-0	Sep 1, 5:39 PM GMT+10 27s	...
✓ Student 3: Market Research Notes & News Feed (Release 0) student-3-ci #1: Commit ddbb637 pushed by SHumzaHussain	feature/student-3-release-0	Sep 1, 5:36 PM GMT+10 20s	...

Brief Description:

1. **Initial CI run:** built and validated the initial Release 0 Implementation of Market Research Notes and News Feed – full CRUD backend HTMX frontend, seeded database and the /ai/briefing PLAN>ACT>OBSERVE>ADAPT endpoint. Tests and docker build passed n first push to feature/student-3-release-0.
2. **Open Pull Request #4:** opened pull request 4 to merge the completed feature branch onto main
3. **Merge Pull request 4:** Merges pull request 4 into main after checks passed completing initial integration of the feature
4. **Resolve merge conflicts with main:** resolved a conflict in the shared docker-compose.yml after Student 2's branch was merged into main reconciling the student 3 service block with the newly added student-4/student-2 service definitions.
5. **Unfied theme redesign (commit):** applied the teams shared monochrome/amber CSS theme and centered layout to the research notes and news feed frontend
6. **Unfied theme redesign (pull request):** opened pull request 6 to merge the shared theme redesign which also touched the shared home page and all students' frontend markup for visual consistency.
7. **Merge pull request 6:** Merged pull request 6 to main completing the visual overhaul of the application

Student-4-ci

Merge pull request #6 from Erfanur1/feature/theme-redesign student-4-ci #7: Commit 76d833a pushed by SHumzaHussain	main	Sep 2, 11:57 PM GMT+10 17s ...
unified theme redesign student-4-ci #6: Pull request #6 opened by SHumzaHussain	feature/theme-redesign	Sep 2, 11:56 PM GMT+10 20s ...
unified theme redesign student-4-ci #5: Commit 987114d pushed by SHumzaHussain	feature/theme-redesign	Sep 2, 11:54 PM GMT+10 26s ...
resolve merge conflicts with main student-4-ci #4: Commit c14d814 pushed by christopherkumarellicherian-bit	Christopher/Student-2/Relea...	Sep 2, 1:10 AM GMT+10 30s ...
Merge pull request #3 from Erfanur1/feature/student-4-release-0 student-4-ci #3: Commit aae4052 pushed by Erfanur1	main	Aug 31, 10:14 PM GMT+10 18s ...
Implement Release 0 for student-4 (AI Market Analyst Assistant) student-4-ci #2: Pull request #3 opened by Erfanur1	feature/student-4-release-0	Aug 31, 10:08 PM GMT+10 22s ...
Implement Release 0 for student-4 (AI Market Analyst Assistant) student-4-ci #1: Commit 799173b pushed by Erfanur1	feature/student-4-release-0	Aug 31, 10:04 PM GMT+10 20s ...

Brief Description:

- Implement Release 0 for student-4** (commit 799173b): Initial commit on feature/student-4-release-0 building the full CRUD backend (markets, analyses, chat_messages), HTMX frontend, seeded database, and the Plan→Act→Observe→Adapt AI endpoints for the AI Market Analyst Assistant.
- Implement Release 0 for student-4** (Pull Request #3 opened): Opened PR #3 to merge feature/student-4-release-0 into main.
- Merge Pull Request #3:** Merged PR #3 into main, integrating the AI Market Analyst Assistant feature.
- Resolve Merge Conflicts with Main:** Resolved merge conflicts introduced when Student 2's branch was merged into main.
- Unified Theme Redesign** (commit): Applied the shared CSS theme (theme.css) to the AI Market Analyst frontend for visual consistency with the rest of the app
- Unified Theme Redesign** (Pull Request #6 opened): Opened PR #6 for the shared theme redesign
- Merge Pull Request #6:** Merged PR #6 into main, completing Release 0 styling for Student 4's feature

12. Docker Compose Execution Evidence

```
(.venv) C:\Users\camel\OneDrive\Desktop\ADV Software Dev\ADVSD\prediction-market-hub 2>docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
advsd-ai-mode-1	advsd-ai-mode	"python app.py"	ai-mode	53 seconds ago	Up 50 seconds	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
advsd-home-1	advsd-home	"python -m http.serv..."	home	53 seconds ago	Up 51 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
advsd-ollama-1	ollama/ollama:latest	"/bin/ollama serve"	ollama	2 hours ago	Up 51 seconds	0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp
advsd-student-1-backend-1	advsd-student-1-backend	"sh -c 'python backe..."	student-1-backend	53 seconds ago	Up 50 seconds	0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
advsd-student-1-frontend-1	advsd-student-1-frontend	"python frontend/app..."	student-1-frontend	52 seconds ago	Up 49 seconds	0.0.0.0:5101->5101/tcp, [::]:5101->5101/tcp
advsd-student-2-backend-1	advsd-student-2-backend	"sh -c 'python backe..."	student-2-backend	53 seconds ago	Up 50 seconds	0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
advsd-student-2-frontend-1	advsd-student-2-frontend	"python frontend/app..."	student-2-frontend	52 seconds ago	Up 50 seconds	0.0.0.0:5102->5102/tcp, [::]:5102->5102/tcp
advsd-student-3-backend-1	advsd-student-3-backend	"sh -c 'python backe..."	student-3-backend	53 seconds ago	Up 50 seconds	0.0.0.0:5003->5003/tcp, [::]:5003->5003/tcp
advsd-student-3-frontend-1	advsd-student-3-frontend	"python frontend/app..."	student-3-frontend	52 seconds ago	Up 50 seconds	0.0.0.0:5103->5103/tcp, [::]:5103->5103/tcp
advsd-student-4-backend-1	advsd-student-4-backend	"sh -c 'python backe..."	student-4-backend	53 seconds ago	Up 50 seconds	0.0.0.0:5004->5004/tcp, [::]:5004->5004/tcp
advsd-student-4-frontend-1	advsd-student-4-frontend	"python frontend/app..."	student-4-frontend	52 seconds ago	Up 50 seconds	0.0.0.0:5104->5104/tcp, [::]:5104->5104/tcp

```
(.venv) C:\Users\camel\OneDrive\Desktop\ADV Software Dev\ADVSD\prediction-market-hub 2>
```

13. Agentic Loop Workflow Record

Record of prompts and AI-agentic review activity used during development (per student).

Student	Prompt / Task	AI Tool Used	Outcome / Review Notes
Student 1	Review my Flask CRUD endpoints in student-1/backend/app.py for correctness and error handling for markets and watchlist routes.	Claude	Confirmed 404 handling on missing markets/watchlist ids; suggested returning JSON error bodies consistently across all routes (applied).
Student 1	Why is my GitHub Actions workflow not triggering when I push to student-1/? Help me fix the paths filter and file location.	Claude	Identified that .github/workflows/ was not at the repository root and that the paths filter didn't match the actual folder name; moved the workflow file and corrected the paths filter — CI now triggers reliably (see Section 11, runs #3–#5).
Student 1	Design an agentic_trace structure for my /ai/trending endpoint that visibly shows the Plan → Act → Observe → Adapt loop, including a graceful fallback if AI-Mode is unreachable.	Claude	Implemented a trace list appended at each stage (Plan/Act/Observe/Adapt) and a fallback message when AI-Mode times out or returns no output, matching the Non-Functional Requirement in Section 2.4.
Student 1	Help me write pytest tests for the watchlist CRUD endpoints and confirm the seeded database has 10+ records per table.	Claude	Generated a pytest fixture that builds a temporary SQLite DB from schema.sql + seed.sql and added assertions for seeded row counts and full watchlist CRUD (see student-1/tests/test_api.py).
Student 2	Why is my docker compose up command failing with a 'docker is not recognized' error, and how do I get the local LLM environment running?	Gemini	<i>Diagnosed a missing Windows Docker Desktop installation, troubleshooted the daemon engine startup sequence, and successfully spun up all 11 microservice containers.</i>
Student 2	Review my backend positions route and database deletion logic; I'm getting an unhandled IntegrityError when trying to close a position that has existing trade history.	Gemini	Identified a missing ON DELETE CASCADE constraint in schema.sql and fixed a route mismatch between the frontend and backend API, resolving the crash.
Student 2	"Help me document my exact SQLite schema and ERD for Section 2.7 based on my actual portfolios, positions, and trade_history tables instead of the outdated registration form.	Gemini	Generated accurate 1NF logical/physical data models and an Entity Relationship Diagram confirming cascading deletions for the final report.
Student 2	"Update my /ai/analyze-risk endpoint to match the team's required AI architecture."	Gemini	Implemented the mandatory Plan → Act → Observe → Adapt (agentic_trace) JSON response structure so the frontend could render the AI's thought process.

Student 3	"My backend is returning sqlite3.operationalerror: unable to open database file on every request even though the container starts fine."	Claude	Diagnosed that the student-3 service in docker-compose.yml was missing its data volume and wasn't running init_db.py before the flask app added the volume and corrected the startup command resolving the error
Student 3	Docker Desktop failed to start with Virtualisation support not detected on first install before containers could be run	Claude	Diagnosed this as either BIOS-level virtualization being disabled or Windows' WSL2/Virtual Machine Platform features not being enabled. Walked through checking Task Manager's CPU tab for virtualization status, enabling Intel VT-x/AMD-V in BIOS if needed, and running <code>dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux</code> and <code>...VirtualMachinePlatform</code> via an admin PowerShell, followed by <code>wsl --install</code> . After a restart, Docker Desktop started successfully ("Engine running"), unblocking all subsequent local development and testing.
Student 4	"Design the /ai/analyze and /ai/chat endpoints so the AI Market Analyst Assistant visibly implements Plan → Act → Observe → Adapt, with a graceful fallback if AI-Mode is unreachable."	Claude	Implemented the <code>_call_ai_mode</code> helper and an <code>agentic_trace</code> list appended at each stage; verified the fallback returns a plain-text message instead of a 500 when AI-Mode times out or is unreachable.
Student 4	"Write pytest tests for the markets/analyses CRUD endpoints and confirm the seeded database has 10+ records per table."	Claude	Generated a pytest fixture (<code>tests/test_api.py</code>) that builds a temporary SQLite DB from <code>schema.sql</code> + <code>seed.sql</code> , with assertions on seeded row counts for all three tables plus full CRUD coverage for markets and analyses.

14. Known Issues and Limitations

- Database architecture: all four features currently use an embedded SQLite database inside the backend container, rather than a separate database-API service as described in Section 7.1 of the specification. This should be reviewed with the tutor and, if required, split into a third container per feature before final submission.
- Student 1 (Watchlist & Discovery): fully functional; no outstanding issues. Cross-feature access still relies on other students calling GET <http://student-1-backend:5001/markets> and /watchlist rather than reading student1.db directly, per Section 7.1 — this should be confirmed with Students 2–4 as their features are integrated.
- Student 2 (Portfolio & Position Tracker): the previous report draft noted missing `DB_PATH/AI_MODE_URL` environment variables and no ai-mode dependency in docker-compose.yml. The current docker-compose.yml shows these configured consistently with the

other students' services — this should be re-verified by Student 2 with a clean `docker compose up` before submission, and this note updated to "resolved" once confirmed.

- **Partial CRUD Implementation (MVP Limitation):** While the Read and Delete operations (utilizing cascading database deletions) are fully functional end-to-end, the frontend currently lacks the UI forms required to create new positions or view historical trade logs. The tracker currently relies on seeded mock data to demonstrate the functional data flow. These expanded user interface components are scoped for development in Release 1.
- **Student 3 (Market Research Notes and News Feed):** fully functional one usability bug was found and fixed during testing. Saving an edited note previously wiped its title and tags because the form only submitted the note content field. Fixed by including title and tags in the edit form and the update request. Like the other features this service currently uses an SQLite database inside its backend container rather than a separate database API container.
- **Student 4 (AI Market Analyst Assistant):** fully functional; uses the same embedded-SQLite pattern as the other three features (see the general database-architecture note above). The `/ai/chat` endpoint currently retains only the last 4 messages of history per market, so very long conversations may lose earlier context — acceptable for Release 0, worth revisiting in Release 1.

15. GitHub Commit Logs & Contribution Logs

15.1 GitHub Commit Log Summary



15.2 Individual Contribution Log

Student	Key Contributions	Commits (approx.)	Attendance Checkpoints
Student 1	CODE: ❑ Built the reference implementation feature (Watchlist & Discovery). This includes in the student 1 folder: – Developed the full CRUD backend. – Developed the HTMX frontend. – Created and seeded the database. – Integrated AI Mode with a visible Plan → Act → Observe → Adapt trace. – Developed an independent CI/CD workflow for the feature ❑ Drafted git repository and its structure ❑ Drafted shared docker-compose.yml. ❑ Diagnosed and fixed a repository structure and CI path issue affecting the team's workflow triggers.	18	3

	Individual Report Sections: ☐ (SECTION 1): Authored Project Overview ☐ (SECTION 2.1): Developed the Agile plan. ☐ (SECTION 2.3) Developed the overall project plan. ☐ (SECTION 3): Repository Structure ☐ (SECTION 5): Created the integrated architecture diagram. ☐ (SECTION 6): Created the Docker Compose architecture diagram. ☐ (SECTION 8.1): AI-Mode Implementation ☐ (SECTION 12): Docker Compose Execution Evidence Shared Report Sections: • (SECTION 2.2): Product Backlog • (SECTION 2.4): Functional & Non-Functional Requirements • (SECTION 2.5): Individual Feature Plan • (SECTION 2.6): Individual Risk Managemnt Plan • (SECTION 2.7): Data Design • (SECTION 4): Individual Software Architecture • (SECTION 8.2): Plan → Act → Observe → Adapt Loop • (SECTION 9): Github Action Workflows • (SECTION 11): Github Actions Workflow Evidence • (SECTION 13): Agentic Loop Workflow Record • (SECTION 14): Known Issues and Limitations • (SECTION 15.2): Individual Contribution Log		
Student 2	CODE: ☐ Built the reference implementation feature (Portfolio & Position Tracker). This includes in the student 2 folder: – Developed the full CRUD backend. – Developed the HTMX frontend.\ – Created and seeded the database. – Integrated AI Mode with a visible Plan → Act → Observe → Adapt trace. – Developed an independent CI/CD workflow for the feature Shared Report Sections: • (SECTION 2.2): Product Backlog • (SECTION 2.4): Functional & Non-Functional Requirements • (SECTION 2.5): Individual Feature Plan • (SECTION 2.6): Individual Risk Managemnt Plan • (SECTION 2.7): Data Design • (SECTION 4): Individual Software Architecture • (SECTION 8.2): Plan → Act → Observe → Adapt Loop • (SECTION 9): Github Action Workflows • (SECTION 11): Github Actions Workflow Evidence	4	3

	• (SECTION 13): Agentic Loop Workflow Record • (SECTION 14): Known Issues and Limitations • (SECTION 15.2): Individual Contribution Log		
Student 3	CODE: ☐ Built the reference implementation feature (Market Research Notes & News Feed). This includes in the student 3 folder: – Developed the full CRUD backend. – Developed the HTMX frontend.\ – Created and seeded the database. – Integrated AI Mode with a visible Plan → Act → Observe → Adapt trace. – Developed an independent CI/CD workflow for the feature • Design the Shared CSS theme for the index and across all features Individual Report Sections: ☐ (SECTION 7): DevOps Pipeline Architecture DevOps Pipeline Architecture ☐ (SECTION 8.3): Plan → Act → Observe → Adapt workflow diagram Shared Report Sections: • (SECTION 2.2): Product Backlog • (SECTION 2.4): Functional & Non-Functional Requirements • (SECTION 2.5): Individual Feature Plan • (SECTION 2.6): Individual Risk Managemnt Plan • (SECTION 2.7): Data Design • (SECTION 4): Individual Software Architecture • (SECTION 8.2): Plan → Act → Observe → Adapt Loop • (SECTION 9): Github Action Workflows • (SECTION 11): Github Actions Workflow Evidence • (SECTION 13): Agentic Loop Workflow Record • (SECTION 14): Known Issues and Limitations • (SECTION 15.2): Individual Contribution Log	2	1
Student 4	CODE: ☐ Built the reference implementation feature (AI Markewt Analyst Assistant). This includes in the student 4 folder: – Developed the full CRUD backend. – Developed the HTMX frontend.\ – Created and seeded the database. – Integrated AI Mode with a visible Plan → Act → Observe → Adapt trace. – Developed an independent CI/CD workflow for the feature	7	3

	☐ Finalised Git repository ad structure Individual Report Sections: ☐ Section 2.4 (Student 4 row): Functional & Non-Functional Requirements ☐ Section 2.5 (Student 4): Individual Feature Plan ☐ Section 2.7 (Student 4): Data Design ☐ Section 4 (Student 4 diagram): Individual Software Architecture ☐ Section 8.2 (Student 4 note): Plan → Act → Observe → Adapt Loop ☐ Section 11 (Student-4-ci): GitHub Actions Workflow Evidence ☐ Section 13 (Student 4 rows): Agentic Loop Workflow Record Shared Report Sections: • (SECTION 2.2): Product Backlog • (SECTION 2.4): Functional & Non-Functional Requirements • (SECTION 2.5): Individual Feature Plan • (SECTION 2.6): Individual Risk Managemnt Plan • (SECTION 2.7): Data Design • (SECTION 4): Individual Software Architecture • (SECTION 8.2): Plan → Act → Observe → Adapt Loop • (SECTION 9): Github Action Workflows • (SECTION 11): Github Actions Workflow Evidence • (SECTION 13): Agentic Loop Workflow Record • (SECTION 14): Known Issues and Limitations • (SECTION 15.2): Individual Contribution Log		
--	--	--	--

16. Showcase Video

Video URL: https://youtu.be/_RuO_whSoN8

